

Laboratoire 1: Détection de l'apnée du sommeil à partir des caractéristiques HRV extraites du signal ECG

Objectif général du laboratoire

Ce laboratoire vise à détecter automatiquement les épisodes d'apnée du sommeil à l'aide du signal ECG uniquement, en extrayant des informations telles que la variabilité de la fréquence cardiaque (HRV). Nous travaillerons avec la base de données Apnea-ECG, qui comprend 70 enregistrements de polysomnographie (PSG), disponibles sur PhysioNet :

 <https://physionet.org/content/apnea-ecg/1.0.0/>

Description des fichiers

Chaque enregistrement (nommé a0x, b0x, etc.) comprend :

- a01.dat → Données ECG (échantillonnées à 100 Hz)
- a01.heu → Fichier d'en-tête contenant les métadonnées
- a01.apn → Annotations binaires (1 par minute) indiquant la présence ou non d'apnée :
 - 'N' : normal
 - 'A' : apnée
 - 'C' : correction (rare)

 Vous pouvez charger ces fichiers en ligne avec la bibliothèque **wfdb** (Python) sans téléchargement local.

Remarque : Les étiquettes d'apnée sont attribuées par des experts humains sur la base de la respiration enregistrée simultanément et des signaux associés.

Rappel : Qu'est-ce qu'un PSG ?

La polysomnographie (PSG) est un enregistrement simultané de plusieurs signaux physiologiques pendant le sommeil : EEG, ECG, EMG, respiration, saturation O₂, etc. Nous nous concentrerons ici uniquement sur le signal ECG.

Apnée du sommeil et ECG

L'apnée du sommeil provoque des interruptions temporaires de la respiration, qui entraînent des changements détectables dans le rythme cardiaque. Ces changements peuvent être observés via :

- Des fluctuations du rythme cardiaque
- Des modifications de la variabilité de la fréquence cardiaque (HRV)
- Des changements dans la forme des ondes ECG

Exercice 1 – Visualisation et filtrage du signal ECG (15%)

1.1. Afficher un segment normal

1. Chargez l'enregistrement a01 depuis PhysioNet avec wfdb.rdsamp.
2. Affichez le deuxième segment d'une minute du signal ECG.
3. Récupérez son annotation associée et identifiez s'il s'agit d'un épisode normal ou apnéique.

1.2. Trouver et comparer le premier épisode d'apnée

1. Parcourez les annotations pour trouver la **première minute étiquetée 'A'**.
2. Affichez le segment ECG correspondant.
3. Comparez visuellement avec le segment normal précédent. Expliquez les différences que vous observez entre le segment d'apnée et le segment normal.

1.3. Appliquer un filtre passe-bande de Butterworth

1. Appliquez un filtre Butterworth passe-bande d'ordre 4, avec des fréquences de coupure de 0.1 Hz à 35 Hz, aux deux segments (normal et apnée).
2. Observez et expliquez comment le filtrage clarifie le signal.
3. Observez et expliquez comment le filtrage modifie la position ou la forme des pics R.

Remarque 1 : Un filtre Butterworth est un type de filtre de signal qui supprime les fréquences indésirables tout en conservant les fréquences souhaitées sans distorsion. Un filtre d'ordre 4 signifie qu'il présente une netteté modérée dans la séparation de ces fréquences.

Remarque 2 : Utilisez la fonction `filtfilt` pour éviter le déphasage du signal. Expliquez comment `filtfilt` évite cela.

Remarque 3 : Vous pouvez observer une distorsion au début ou à la fin du signal après le filtrage. Cela est causé par des effets de **bords** dus au fonctionnement du filtre, qui ne dispose pas d'informations « au-delà » des extrémités du signal pour effectuer le calcul.

 Une **solution** est d'ajouter du **padding** (remplissage) avant de filtrer, en prolongeant artificiellement le signal par réflexion de ses premières et dernières secondes. Cette méthode aide le filtre à se « stabiliser » au bord du signal, réduisant les distorsions introduites à ces endroits.

1.4. Fréquence d'échantillonnage minimale

Sachant que le signal est initialement à **100 Hz**, proposez une **fréquence d'échantillonnage minimale acceptable après filtrage**. Justifiez votre réponse en fonction du **théorème de Nyquist**.

💡 Aide – Charger le signal

```
import wfdb
signal, fields = wfdb.rdsamp(record_name, pn_dir=pn_dir)
```

💡 Aide – Chargement des données avec wfdb

Voici un exemple de code pour charger le signal ECG et les annotations d'apnée depuis PhysioNet :

```
import wfdb
import matplotlib.pyplot as plt

# Nom de l'enregistrement
record_name = 'a01'
pn_dir = 'apnea-ecg'

# Étape 1 : Charger le signal ECG (local ou en ligne)
signal, fields = wfdb.rdsamp(record_name, pn_dir=pn_dir)

# Extraire la dérivation ECG (un seul canal)
ecg = signal[:, 0]
fs = fields['fs'] # Fréquence d'échantillonnage (devrait être 100 Hz)

# Étape 2 : Charger les annotations d'apnée
annotation = wfdb.rdann(record_name, 'apn', pn_dir=pn_dir)
```

Utilisez `annotation.symbol` pour accéder aux étiquettes minute par minute (par ex. 'N' = normal, 'A' = apnée).

Chaque segment correspond à une durée de **60 secondes**, soit **6000 échantillons** à 100 Hz.

💡 Aide – Conception d'un filtre passe-bande Butterworth

```
from scipy.signal import butter, filtfilt

# -----
# Design Butterworth Bandpass Filter
# -----
def butter_bandpass(lowcut, highcut, fs, order=4):
    nyq = 0.5 * fs # Nyquist frequency
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return b, a
```

🧪 Exercice 2 – Extraction de caractéristiques HRV (20%)

Dans cet exercice, nous allons extraire des caractéristiques du rythme cardiaque (HRV) à partir des segments ECG filtrés (normal et apnée) pour les utiliser plus tard dans une classification automatique.

2.1. Importance des caractéristiques

Expliquez pourquoi l'extraction de caractéristiques facilite l'apprentissage automatique à partir de signaux physiologiques comme l'ECG.

2.2. Extraction des pics R

Pour extraire les caractéristiques HRV, il est essentiel de détecter les pics R dans le signal ECG, puis de calculer les intervalles R-R.

 Nous recommandons d'utiliser les fonctions suivantes de la bibliothèque biosppy :

```
from biosppy.signals.ecg import hamilton_segmenter, correct_rpeaks
```

Que font ces fonctions ?

- **hamilton_segmenter()** :
Applique l'algorithme de détection de QRS de Hamilton. Il détecte les positions initiales des pics R dans le signal ECG.
- **correct_rpeaks()** :
Corrige les pics R détectés en fonction du signal d'origine, pour supprimer les faux positifs ou ajuster la position du pic précisément au maximum local.

À faire

Tracez à nouveau les deux signaux avec les pics R marqués par des points. Utilisez la fonction `hamilton_segmenter()` ; si nécessaire, vous pouvez corriger les pics avec `correct_rpeaks()`.

2.3. Caractéristiques HRV

Les caractéristiques que nous allons extraire sont des caractéristiques de variabilité du rythme cardiaque (HRV). Elles seront calculées à partir des intervalles R-R extraits du signal ECG filtré.

◆ Caractéristiques temporelles (Time-domain)

Ces caractéristiques sont directement calculées à partir des intervalles R-R (en millisecondes) :

- SDNN : écart-type des intervalles R-R (variabilité globale)
- RMSSD : racine carrée de la moyenne des carrés des différences successives
- SDSD : écart-type des différences successives
- pNN50 : pourcentage de paires d'intervalles R-R dont la différence est supérieure à 50 ms

♦ Caractéristiques fréquentielles (Frequency-domain)

Ces caractéristiques sont obtenues par une analyse spectrale des intervalles R-R, à partir de la densité spectrale de puissance (PSD) estimée via la méthode de Welch ou une transformée de Fourier rapide (FFT) :

- Total Power : puissance totale du spectre HRV
- VLF : puissance dans la bande très basse fréquence (0–0.04 Hz)
- LF : puissance dans la bande basse fréquence (0.04–0.15 Hz)
- HF : puissance dans la bande haute fréquence (0.15–0.4 Hz)

✅ Fonctions à utiliser

Pour extraire ces caractéristiques, on utilisera la bibliothèque `hrvanalysis` :

```
from hrvanalysis import get_time_domain_features, get_frequency_domain_features
```

- `get_time_domain_features(rr_intervals)` : retourne un dictionnaire contenant les caractéristiques temporelles.
- `get_frequency_domain_features(rr_intervals)` : retourne un dictionnaire contenant les caractéristiques fréquentielles.

Les intervalles R-R doivent être donnés en millisecondes sous forme de liste Python.

👉 Pour utiliser `hrvanalysis`, vous devez installer **Git** (<https://git-scm.com/downloads/win>), car ce module est hébergé sur **GitHub**, et pip a besoin de Git pour le télécharger.

✍️ À faire

1. Appliquez ces deux fonctions aux intervalles R-R extraits pour le segment normal et le segment avec apnée. Affichez les résultats sous forme de tableau comparatif avec les colonnes suivantes :
 - Nom de la caractéristique
 - Valeur (segment normal)
 - Valeur (segment apnée)
2. Comparez les deux segments et interprétez les différences observées.

Exercice 3 – Conception d'un classifieur MLP (Perceptron Multi-Couche) (40%)

Concevoir un petit modèle de **réseau de neurones (MLP)** pour **classifier automatiquement** les segments de sommeil en deux classes :

- **Apnée**
- **Normal**

Les caractéristiques **HRV** (SDNN, RMSSD, etc.) sont déjà extraites à partir des signaux ECG filtrés.

 Vous les trouverez dans un fichier **CSV**.

Données fournies

Un fichier **features_apnea.csv** contenant :

- Une ligne par segment ECG (1 minute)
- Une colonne par caractéristique HRV
- Une colonne label contenant les classes (0 = normal, 1 = apnée) :
 - 0 → Normal
 - 1 → Apnée

3.1 – Chargement et division du jeu de données

1. Chargez le fichier .csv à l'aide de pandas.
2. Séparez les données en **caractéristiques (X)** et **étiquettes (y)**.
3. Divisez les données en trois sous-ensembles :
 - **Entraînement** : 70 %
 - **Validation** : 15 %
 - **Test** : 15 %

 Utilisez `train_test_split` de `sklearn.model_selection` avec l'option `stratify=y` pour garantir une répartition équilibrée des classes dans chaque ensemble.

3.2 – Création d'un modèle MLP simple

Utilisez `sklearn.neural_network.MLPClassifier` pour construire un petit perceptron multicouche.

Hyperparamètres à tester (sur l'ensemble de validation) :

- `hidden_layer_sizes` : nombre de neurones cachés (exemples : (10,), (20,), (20, 10))
- `activation` : fonction d'activation (exemples : 'relu', 'tanh')
- `alpha` : coefficient de régularisation L2 (exemples : 0.0001, 0.001, 0.01)
- Cela aide à éviter l'overfitting en pénalisant les poids trop grands.

🔒 Hyperparamètres fixés :

- max_iter = 500 (Nombre maximal d'itérations $\approx$ nombre d'époques)
- solver = 'adam' (méthode d'optimisation adaptative)
- random_state = 42 (initialisation reproductible)

! Pas de recherche automatique des hyperparamètres. Les étudiants doivent tester **manuellement** au moins 6 combinaisons différentes et comparer les performances.

👉 3.3 – Évaluation du modèle

1. Pour chaque configuration testée, évaluez :
 - **Accuracy** sur les ensembles de train, validation et test
 - **Matrice de confusion** sur les ensembles de train, validation et test
 - **Courbe ROC et AUC** (*facultatif*)
2. Choisissez la **meilleure configuration** selon l'accuracy sur l'ensemble de validation.

👉 3.4 – Interprétation et analyse

Pour chaque modèle :

- Le modèle est-il **sous-appris** (underfitting) ou **sur-appris** (overfitting) ?
- Les performances sur l'ensemble test sont-elles cohérentes avec celles de validation ?
- Quelle configuration donne les meilleurs résultats ? (sur la base de validation)
- Que peut-on conclure sur l'efficacité des **caractéristiques HRV** pour détecter l'apnée ?

💡 Astuce : Exemple de code de départ

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt

# Charger les données
df = pd.read_csv('features_apnea.csv')
X = df.drop('label', axis=1)
y = df['label']

# Séparation des données
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, stratify=y,
                                                    random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
                                                stratify=y_temp, random_state=42)

# Création du modèle
mlp = MLPClassifier(hidden_layer_sizes=(20,), activation='relu', alpha=0.001,
```

```

max_iter=300, solver='adam', random_state=42)

# Entraînement
mlp.fit(X_train, y_train)

# Évaluation
val_acc = accuracy_score(y_val, mlp.predict(X_val))
test_acc = accuracy_score(y_test, mlp.predict(X_test))

print("Validation Accuracy:", val_acc)
print("Test Accuracy:", test_acc)
print("Matrice de confusion (test) :\n", confusion_matrix(y_test, mlp.predict(X_test)))

```

Exercice 4 – Courbes d'apprentissage (25%)

Dans cet exercice, nous voulons observer le surajustement et trouver un nombre d'époques permettant de l'éviter.

4.1 – Courbes d'entraînement et validation

Pour une configuration MLP :

1. Tracez la courbe de perte (loss) sur l'ensemble d'entraînement et la validation.
💡 Utilisez `warm_start=True` et `max_iter=1` dans une boucle pour entraîner votre MLP époque par époque.

4.2 – Analyse des courbes

1. À quelle époque survient l'overfitting (écart entre train/val) ?
Si vous ne constatez pas de surajustement, vous devez modifier votre modèle. Voir remarque 2.
2. À quel moment aurait-il fallu arrêter l'entraînement (early stopping) ?
3. À quelles époques le modèle est-il sous-ajusté et à quelles époques est-il surajusté ?
4. Vos observations sont-elles cohérentes avec les résultats sur le test ?

Remarque 1:

Le Problème des Fluctuations (Le Bruit)

Les courbes d'apprentissage (comme l'accuracy ou l'erreur de votre modèle au fil du temps) sont souvent "bruyantes", avec beaucoup de montées et de descentes. Pourquoi ce "bruit" ?

1. **Entraînement par Mini-Lots (Mini-Batch Training)** : Lorsque vous utilisez des optimiseurs comme **Adam** (ce qui est le cas dans votre code), le réseau de neurones n'apprend pas en traitant *toutes* les données d'entraînement d'un coup. Il les découpe en petits morceaux appelés **mini-**

lots. Chaque mini-lot est un échantillon légèrement différent, et la performance du modèle peut "sauter" un peu à chaque mise à jour basée sur un nouveau mini-lot, créant le bruit que vous voyez.

2. **Nature Stochastique** : Le processus d'optimisation lui-même contient une part de hasard (c'est le sens de "stochastique" dans des algorithmes comme la Descente de Gradient Stochastique ou Adam). Ce hasard aide le modèle à ne pas rester bloqué dans de mauvaises solutions, mais il contribue aussi aux légères oscillations.

Ce bruit rend difficile de voir la **vraie tendance** de l'apprentissage de votre modèle.

La Solution : Le Lissage (Smoothing)

Pour obtenir une image plus claire et voir la progression réelle, nous **lissons** les courbes.

1. **Moyenne Mobile Exponentielle (EMA)** : C'est la technique courante que vous utilisez. Elle calcule un nouveau point lissé en prenant une moyenne pondérée entre le point brut actuel et le *point lissé précédent*. Cela "filtre" efficacement les petites variations à court terme pour révéler la direction générale.

```
def smooth_curve(points, factor=0.9):
    smoothed = []
    for p in points:
        if smoothed:
            smoothed.append(smoothed[-1] * factor + p * (1 - factor))
        else:
            smoothed.append(p)
    return smoothed
```

2. Ajuster la Taille des Lots (batch_size) :

- **Comment** : Dans MLPClassifier, vous pouvez contrôler directement la taille des lots en ajoutant le paramètre batch_size. Par défaut, il est sur 'auto' (généralement min(200, n_samples)).

```
mlp = MLPClassifier(
    # ... autres paramètres ...
    solver='adam',
    batch_size=32, # Ou 64, 128, etc. - à expérimenter !
    # ...
)
```

- **Impact** :

- **Petit batch_size** : Plus de mises à jour, plus de bruit (car les mises à jour sont basées sur moins de données), mais peut aider le modèle à trouver de meilleures solutions et à mieux généraliser.
- **Grand batch_size** : Moins de mises à jour, courbes d'entraînement plus lisses, entraînement potentiellement plus rapide par époque, mais le modèle risque de se bloquer dans des solutions moins optimales et parfois de moins bien généraliser. Vous pourriez aussi avoir besoin d'ajuster le taux d'apprentissage.

👉 Remarque 2 : Le Surapprentissage (Overfitting) :

Il est probable que vous ne puissiez pas voir le surajustement dans vos modèles de l'exercice 3. Voici la solution :

Si nous **augmentons la complexité du réseau** (par exemple, en ajoutant plus de couches (par exemple, >2) et de neurones (par exemple, (100, 20, 10)), **réduisons ou supprimons le terme de régularisation alpha** (qui aide à prévenir le surapprentissage en pénalisant les poids trop grands), **et/ou augmentons le nombre d'époques d'entraînement** (par exemple, 2000), nous pourrions alors commencer à observer clairement les signes de surapprentissage. Dans ce cas, la courbe de performance sur l'ensemble d'entraînement continuerait de s'améliorer, tandis que celle sur l'ensemble de validation commencerait à stagner ou même à se détériorer, indiquant que le modèle apprend les spécificités de l'entraînement au détriment de sa capacité à généraliser sur de nouvelles données.

🧪 Exercice 5 – Arrêt Anticipé (Early Stopping) (Bonus +10%)

Implémenter l'arrêt anticipé pour optimiser l'entraînement de votre réseau en utilisant la perte de validation (val_loss) :

1. Modifiez votre code :
 - Arrêtez l'entraînement si la val_loss n'a pas diminué pendant 150 époques (patience = 150).
 - Définissez epochs = 1500 (max) et min_epochs = 20.
 - (*Rappel : La log_loss réelle est souvent meilleure pour l'early stopping.*)
2. Exécutez le code.
3. Tracez les courbes lissées de train_loss et val_loss.
4. Analysez vos résultats :
 - Quand l'entraînement s'est-il arrêté et pourquoi ? Expliquez comment la patience a été atteinte. Quel est l'avantage ?